ATELIER COMPLET

CI/CD avec GitHub Actions

Spring Boot · Maven · Docker

🐳 **Dockerfile multi-stage**
⚙️ **Pipeline GitHub Actions (deploy.yml)**
🚀 **Déploiement automatisé sur serveur distant**

Durée estimée : 3 à 4 heures | Niveau : Intermédiaire

1. Introduction & Objectifs

Cet atelier vous guide pas à pas dans la mise en place d'un pipeline CI/CD complet pour une application Java Spring Boot construite avec Maven. Vous partirez du dépôt de référence eldiopo/spring-maven et vous le transformerez en une chaîne de livraison continue entièrement automatisée.

Objectifs pédagogiques

- Comprendre l'architecture d'un pipeline CI/CD moderne
- Créer un Dockerfile multi-stage optimisé pour une application Spring Boot
- Écrire un workflow GitHub Actions complet (build, test, push, deploy)
- Sécuriser les secrets et variables d'environnement dans GitHub
- Déployer automatiquement sur un serveur distant via SSH

Stack technologique

Composant	Technologie	Rôle
Application	Spring Boot 3.x	Framework Java pour API REST
Build	Apache Maven 3.9	Gestion des dépendances & compilation
Conteneur	Docker 24+	Packaging & isolation de l'application
CI/CD	GitHub Actions	Automatisation du pipeline
Registry	Docker Hub / GHCR	Stockage des images Docker
Déploiement	SSH + Docker Compose	Mise en production sur serveur

2. Structure du Projet

Le projet spring-maven est une application Spring Boot standard exposant une API REST. Voici l'arborescence que vous devez avoir (ou obtenir après clonage du dépôt) :

📁 **Arborescence du projet**

```
spring-maven/
├── .github/
│   └── workflows/
│       └── deploy.yml          # ← Pipeline CI/CD (créé dans cet atelier)
├── src/
│   ├── main/
│   │   ├── java/com/example/
│   │   │   └── SpringMavenApplication.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
│       └── java/com/example/
│           └── SpringMavenApplicationTests.java
├── Dockerfile                  # ← Créé dans cet atelier
├── docker-compose.yml          # ← Optionnel, pour le déploiement
└── pom.xml
```

Contenu clé du pom.xml

Le fichier pom.xml définit la configuration de build Maven. Les points importants pour Docker :

- Packaging en JAR exécutable (fat JAR avec toutes les dépendances)
- Plugin spring-boot-maven-plugin pour le repackaging
- Version Java cible (17 ou 21 recommandé)

📄 **pom.xml (extrait)**

```
<!-- Extrait pom.xml -->
<properties>
    <java.version>17</java.version>
</properties>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

3. Le Dockerfile Multi-Stage

Un Dockerfile multi-stage sépare la phase de build (compilation Maven) de la phase de runtime (exécution du JAR). Cela produit une image finale beaucoup plus légère, car les outils de compilation ne sont pas inclus dans l'image de production.

> 💡 **Pourquoi multi-stage ?**
> → Image de build (Maven + JDK) : ~600 MB
> → Image de production (JRE seul) : ~180 MB
> → Gain : jusqu'à 70% de réduction de taille

3.1 Le Dockerfile complet

Créez le fichier Dockerfile à la racine du projet :

🐳 **Dockerfile**

```
# ============================================================
# STAGE 1 : BUILD
# Utilise une image Maven avec JDK pour compiler l'application
# ============================================================
FROM maven:3.9-eclipse-temurin-17 AS builder

# Répertoire de travail dans le conteneur de build
WORKDIR /app

# Copier d'abord le pom.xml pour exploiter le cache Docker
# Si pom.xml ne change pas, les dépendances ne sont pas re-téléchargées
COPY pom.xml .

# Télécharger toutes les dépendances (sans compiler le code)
RUN mvn dependency:go-offline -B

# Copier le code source
COPY src ./src

# Compiler et packager l'application (skip tests → exécutés en CI)
RUN mvn clean package -DskipTests -B

# ============================================================
# STAGE 2 : RUNTIME
# Image légère avec seulement le JRE (sans Maven ni JDK complet)
# ============================================================
FROM eclipse-temurin:17-jre-alpine AS runtime

# Métadonnées de l'image
LABEL maintainer="votre-email@exemple.com"
LABEL version="1.0"
LABEL description="Spring Boot Application"
```

```
# Créer un utilisateur non-root pour la sécurité
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# Répertoire de travail
WORKDIR /app

# Copier UNIQUEMENT le JAR depuis le stage de build
COPY --from=builder /app/target/*.jar app.jar

# Changer le propriétaire des fichiers vers l'utilisateur non-root
RUN chown -R appuser:appgroup /app

# Utiliser l'utilisateur non-root
USER appuser

# Exposer le port Spring Boot (défaut : 8080)
EXPOSE 8080

# Variables d'environnement par défaut
ENV JAVA_OPTS="-Xms256m -Xmx512m -XX:+UseContainerSupport"
ENV SPRING_PROFILES_ACTIVE=prod

# Commande de démarrage avec options JVM
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

3.2 Explication des choix

Instruction	Justification
`maven:3.9-eclipse-temurin-17`	Image officielle Maven + Temurin JDK 17 (LTS, recommandé)
`eclipse-temurin:17-jre-alpine`	JRE Alpine = image minimale (~60MB) sans outils superflus
`dependency:go-offline`	Pre-télécharge les deps → cache Docker efficace
`-DskipTests`	Les tests sont exécutés séparément dans le pipeline CI
`adduser -S appuser`	Sécurité : principe du moindre privilège (pas de root)
`UseContainerSupport`	JVM respecte les limites CPU/mémoire du conteneur
`SPRING_PROFILES_ACTIVE=prod`	Active le profil de configuration de production

3.3 Test local du Dockerfile

Avant de pousser vers GitHub, testez votre image localement :

```
🖥️ Terminal — Test local
# Construction de l'image
```

```
docker build -t spring-maven:latest .

# Vérification de la taille de l'image
docker images spring-maven

# Lancement du conteneur en local
docker run -d \
  --name spring-app \
  -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=dev \
  spring-maven:latest

# Vérification des logs
docker logs -f spring-app

# Test de l'endpoint
curl http://localhost:8080/actuator/health

# Nettoyage
docker stop spring-app && docker rm spring-app
```

4. Le Workflow GitHub Actions (deploy.yml)

Le fichier deploy.yml définit l'intégralité du pipeline CI/CD. Il est déclenché à chaque push sur la branche main et enchaîne automatiquement les étapes de test, build, publication Docker et déploiement sur serveur.

📍 **Emplacement du fichier**
Créez le dossier .github/workflows/ à la racine de votre projet,
puis créez le fichier deploy.yml dans ce dossier.

Commande : mkdir -p .github/workflows && touch .github/workflows/deploy.yml

4.1 Le fichier deploy.yml complet

⚙️ **.github/workflows/deploy.yml**

```yaml
# ============================================================
# deploy.yml — Pipeline CI/CD complet
# Spring Boot + Maven + Docker + SSH Deploy
# ============================================================

name: CI/CD — Build, Test & Deploy

# ──────────────────────────────────────────
# DÉCLENCHEURS
# ──────────────────────────────────────────
on:
  push:
    branches: [ main ]          # Déclenche sur chaque push vers main
  pull_request:
    branches: [ main ]          # Déclenche sur les PR vers main
  workflow_dispatch:            # Permet le déclenchement manuel

# ──────────────────────────────────────────
# VARIABLES D'ENVIRONNEMENT GLOBALES
# ──────────────────────────────────────────
env:
  DOCKER_IMAGE: ${{ secrets.DOCKERHUB_USERNAME }}/spring-maven
  JAVA_VERSION: '17'
  CONTAINER_NAME: spring-maven-app
  APP_PORT: '8080'

# ──────────────────────────────────────────
# JOBS
# ──────────────────────────────────────────
jobs:

  # ----------------------------------------
  # JOB 1 : TESTS
  # ----------------------------------------
```

```yaml
  test:
    name: 🧪 Tests Maven
    runs-on: ubuntu-latest

    steps:
      - name: 📥 Checkout du code
        uses: actions/checkout@v4

      - name: ☕ Setup JDK ${{ env.JAVA_VERSION }}
        uses: actions/setup-java@v4
        with:
          java-version: ${{ env.JAVA_VERSION }}
          distribution: temurin
          cache: maven                    # Cache automatique du repo Maven

      - name: 🧪 Exécution des tests
        run: mvn test -B --no-transfer-progress

      - name: 📊 Publication des résultats de tests
        uses: dorny/test-reporter@v1
        if: always()                      # Publier même en cas d'échec
        with:
          name: Maven Tests
          path: target/surefire-reports/*.xml
          reporter: java-junit

  # ----------------------------------------
  # JOB 2 : BUILD & PUSH DOCKER
  # ----------------------------------------
  build-and-push:
    name: 🐳 Build & Push Docker
    runs-on: ubuntu-latest
    needs: test                           # Attend la réussite des tests
    if: github.ref == 'refs/heads/main'   # Seulement sur main (pas les PR)

    outputs:
      image-tag: ${{ steps.meta.outputs.tags }}
      image-digest: ${{ steps.build.outputs.digest }}

    steps:
      - name: 📥 Checkout du code
        uses: actions/checkout@v4

      - name: 🔑 Connexion à Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}

      - name: 🏷️ Génération des métadonnées (tags & labels)
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.DOCKER_IMAGE }}
          tags: |
```

```
            type=sha,prefix=sha-              # Tag par commit SHA
            type=raw,value=latest             # Tag 'latest' toujours mis à jour
            type=semver,pattern={{version}}  # Tag de version si tag Git

      - name: 🔧 Setup Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: 🏗️ Build & Push de l'image
        id: build
        uses: docker/build-push-action@v5
        with:
          context: .
          file: ./Dockerfile
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha               # Cache GitHub Actions
          cache-to: type=gha,mode=max
          platforms: linux/amd64,linux/arm64 # Multi-architecture

  # ----------------------------------------
  # JOB 3 : DÉPLOIEMENT
  # ----------------------------------------
  deploy:
    name: 🚀 Déploiement Production
    runs-on: ubuntu-latest
    needs: build-and-push
    if: github.ref == 'refs/heads/main'
    environment:
      name: production                      # Environnement GitHub (avec
approbation)
      url: http://${{ secrets.SERVER_HOST }}:${{ env.APP_PORT }}

    steps:
      - name: 🚀 Déploiement via SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SERVER_SSH_KEY }}
          port: ${{ secrets.SERVER_PORT }}  # Défaut 22 si non défini
          script: |
            # Connexion à Docker Hub sur le serveur
            echo '${{ secrets.DOCKERHUB_TOKEN }}' | \
              docker login -u '${{ secrets.DOCKERHUB_USERNAME }}' --password-stdin

            # Arrêt et suppression de l'ancien conteneur
            docker stop ${{ env.CONTAINER_NAME }} 2>/dev/null || true
            docker rm   ${{ env.CONTAINER_NAME }} 2>/dev/null || true

            # Téléchargement de la nouvelle image
            docker pull ${{ env.DOCKER_IMAGE }}:latest

            # Démarrage du nouveau conteneur
            docker run -d \
```

```
            --name ${{ env.CONTAINER_NAME }} \
            --restart unless-stopped \
            -p ${{ env.APP_PORT }}:8080 \
            -e SPRING_PROFILES_ACTIVE=prod \
            -e SPRING_DATASOURCE_URL=${{ secrets.DB_URL }} \
            -e SPRING_DATASOURCE_USERNAME=${{ secrets.DB_USER }} \
            -e SPRING_DATASOURCE_PASSWORD=${{ secrets.DB_PASSWORD }} \
            ${{ env.DOCKER_IMAGE }}:latest

          # Nettoyage des anciennes images
          docker image prune -f

          # Vérification du démarrage (attente 15s)
          sleep 15
          docker ps | grep ${{ env.CONTAINER_NAME }}
          echo '✅ Déploiement réussi !'

      - name: 🔔 Notification de succès
        if: success()
        run: |
          echo '🎉 Application déployée avec succès !'
          echo "Image: ${{ env.DOCKER_IMAGE }}:latest"
          echo "Commit: ${{ github.sha }}"

      - name: 🚨 Notification d'échec
        if: failure()
        run: |
          echo '❌ Le déploiement a échoué !'
          echo "Commit: ${{ github.sha }}"
          echo "Consulter les logs ci-dessus pour diagnostiquer."
```

5. Configuration des Secrets GitHub

Les secrets permettent de stocker des informations sensibles sans les exposer dans le code. Ils sont chiffrés et injectés comme variables d'environnement lors de l'exécution du workflow.

Comment ajouter les secrets ?

Naviguez vers : Settings → Secrets and variables → Actions → New repository secret

Secret	Valeur attendue	Description
`DOCKERHUB_USERNAME`	votre-username	Nom d'utilisateur Docker Hub
`DOCKERHUB_TOKEN`	dckr_pat_xxx...	Token d'accès Docker Hub (pas le mot de passe)
`SERVER_HOST`	192.168.1.100	IP ou domaine du serveur de prod
`SERVER_USER`	ubuntu	Utilisateur SSH sur le serveur
`SERVER_SSH_KEY`	-----BEGIN RSA...	Clé privée SSH (RSA ou Ed25519)
`SERVER_PORT`	22	Port SSH (optionnel, défaut 22)
`DB_URL`	jdbc:postgresql://...	URL JDBC de la base de données
`DB_USER`	dbuser	Utilisateur de la base de données
`DB_PASSWORD`	s3cr3t!	Mot de passe de la base de données

Génération de la clé SSH

Sur votre machine locale, générez une paire de clés SSH dédiée au déploiement :

🔑 Génération des clés SSH

```
# Génération d'une clé Ed25519 (plus moderne et sécurisée que RSA)
ssh-keygen -t ed25519 -C "github-actions-deploy" -f ~/.ssh/deploy_key

# Afficher la clé PUBLIQUE → à ajouter sur le serveur
cat ~/.ssh/deploy_key.pub

# Afficher la clé PRIVÉE → à coller dans le secret SERVER_SSH_KEY
cat ~/.ssh/deploy_key

# Sur le SERVEUR, ajouter la clé publique
echo 'VOTRE_CLE_PUBLIQUE' >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

⚠️ **Sécurité — Bonnes pratiques**

→ Ne jamais commiter des secrets dans le code source

→ Utiliser des tokens Docker Hub (pas le mot de passe du compte)
→ Créer une clé SSH dédiée au CI/CD, révocable indépendamment
→ Utiliser les Environments GitHub pour ajouter une validation manuelle
→ Faire tourner les secrets régulièrement (rotation)

6. Docker Compose pour la Production

Pour une gestion plus robuste du déploiement sur le serveur, vous pouvez utiliser Docker Compose à la place des commandes docker run. Cela facilite la gestion des réseaux, volumes et dépendances entre services.

docker-compose.yml

```yaml
# docker-compose.yml — Production
version: '3.9'

services:
  app:
    image: ${DOCKERHUB_USERNAME}/spring-maven:latest
    container_name: spring-maven-app
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=prod
      - SPRING_DATASOURCE_URL=${DB_URL}
      - SPRING_DATASOURCE_USERNAME=${DB_USER}
      - SPRING_DATASOURCE_PASSWORD=${DB_PASSWORD}
      - JAVA_OPTS=-Xms256m -Xmx512m
    networks:
      - app-network
    healthcheck:
      test: ["CMD", "wget", "-q", "--spider",
             "http://localhost:8080/actuator/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 60s
    depends_on:
      postgres:
        condition: service_healthy

  postgres:
    image: postgres:15-alpine
    container_name: spring-postgres
    restart: unless-stopped
    environment:
      - POSTGRES_DB=springdb
      - POSTGRES_USER=${DB_USER}
      - POSTGRES_PASSWORD=${DB_PASSWORD}
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
      - app-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5
```

```
networks:
  app-network:
    driver: bridge

volumes:
  postgres-data:
    driver: local
```

Adaptation du deploy.yml avec Docker Compose

Si vous utilisez Docker Compose, remplacez la section de déploiement SSH par :

```
⚙️ deploy.yml — extrait avec Docker Compose
      - name: 🚀 Déploiement avec Docker Compose
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${{ secrets.SERVER_HOST }}
          username: ${{ secrets.SERVER_USER }}
          key: ${{ secrets.SERVER_SSH_KEY }}
          script: |
            cd /opt/spring-maven

            # Mise à jour des variables d'environnement
            cat > .env << EOF
            DOCKERHUB_USERNAME=${{ secrets.DOCKERHUB_USERNAME }}
            DB_URL=${{ secrets.DB_URL }}
            DB_USER=${{ secrets.DB_USER }}
            DB_PASSWORD=${{ secrets.DB_PASSWORD }}
            EOF

            # Pull de la nouvelle image
            docker compose pull app

            # Redémarrage avec zero-downtime
            docker compose up -d --no-deps app

            # Nettoyage
            docker image prune -f
```

7. Déroulement du Pipeline

Voici le flux complet de ce qui se passe lorsque vous faites un git push vers la branche main :

#	Étape	Description
1	`git push`	Vous poussez votre code vers GitHub sur la branche main
2	`Déclenchement`	GitHub Actions détecte le push et démarre le workflow
3	`Checkout`	Le runner télécharge le code source du dépôt
4	`Setup JDK`	Installation de Temurin JDK 17 avec cache Maven
5	`Tests Maven`	Exécution de mvn test — si KO, le pipeline s'arrête
6	`Login Docker`	Authentification sur Docker Hub avec les secrets
7	`Build image`	Construction de l'image Docker multi-stage
8	`Push image`	Publication de l'image sur Docker Hub (latest + sha)
9	`SSH Deploy`	Connexion SSH au serveur, pull de la nouvelle image
10	`Redémarrage`	Arrêt ancien conteneur, démarrage du nouveau
11	`Notification`	Rapport de succès ou d'échec dans les logs

8. Dépannage & Problèmes Courants

Problème 1 — L'image Docker ne se build pas

🔴 **Symptôme**
Le stage de build Maven échoue avec des erreurs de dépendances.

✅ **Solution**

```
# Vérifier que le pom.xml est valide
mvn validate

# Forcer le re-téléchargement des dépendances
mvn dependency:purge-local-repository

# Builder sans cache Docker
docker build --no-cache -t spring-maven:latest .
```

Problème 2 — Échec de connexion SSH

🔴 **Symptôme**
ssh: connect to host xxx port 22: Connection refused dans les logs GitHub Actions.

✅ **Solution**

```
# Vérifier la connectivité SSH depuis votre machine
ssh -i ~/.ssh/deploy_key -p 22 user@server-ip

# Vérifier les authorized_keys sur le serveur
cat ~/.ssh/authorized_keys

# Vérifier les permissions
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys

# Vérifier que le service SSH tourne
sudo systemctl status ssh
```

Problème 3 — L'application ne démarre pas

🔴 **Symptôme**
Le conteneur s'arrête immédiatement après le démarrage.

✅ **Solution**

```
# Voir les logs du conteneur
docker logs spring-maven-app

# Voir les 50 dernières lignes
docker logs --tail=50 spring-maven-app
```

```
# Inspecter l'état du conteneur
docker inspect spring-maven-app

# Lancer en mode interactif pour débugger
docker run -it --rm spring-maven:latest /bin/sh
```

Problème 4 — Le pipeline échoue sur les tests

🔴 **Symptôme**
Tests Maven en échec dans GitHub Actions mais qui passent en local.

✅ **Solution**

```
# Vérifier les variables d'environnement manquantes
# Ajouter dans deploy.yml sous la step de test :

      - name: 🧪 Exécution des tests
        run: mvn test -B
        env:
          SPRING_DATASOURCE_URL: ${{ secrets.TEST_DB_URL }}
          # Ou utiliser H2 en mémoire pour les tests

# Dans application-test.properties :
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=create-drop
```

9. Exercices Pratiques

🎯 **Objectif des exercices**
Ces exercices progressifs vous permettent de valider votre compréhension et d'approfondir les concepts abordés dans cet atelier.

Exercice 1 — Mise en place (⭐)

- Clonez le dépôt eldiopo/spring-maven
- Créez le Dockerfile tel que décrit dans la section 3
- Buildez l'image localement et vérifiez qu'elle fonctionne
- Commitez et poussez le Dockerfile vers votre fork

Exercice 2 — Pipeline de base (⭐⭐)

- Créez le workflow GitHub Actions avec uniquement le job de test
- Vérifiez que le pipeline se déclenche sur un push
- Consultez les logs dans l'onglet Actions de votre dépôt
- Intentionnellement cassez un test et observez le comportement

Exercice 3 — Build & Push Docker (⭐⭐)

- Créez un compte Docker Hub et générez un token d'accès
- Ajoutez les secrets DOCKERHUB_USERNAME et DOCKERHUB_TOKEN
- Complétez le workflow avec le job build-and-push
- Vérifiez que l'image apparaît dans votre Docker Hub

Exercice 4 — Déploiement SSH (⭐⭐⭐)

- Provisionnez un serveur Linux (VM, VPS, ou machine locale)
- Installez Docker sur ce serveur
- Générez une paire de clés SSH et configurez l'accès
- Ajoutez tous les secrets nécessaires dans GitHub
- Complétez le workflow avec le job deploy
- Faites un push et vérifiez le déploiement automatique

Exercice 5 — Améliorations avancées (⭐⭐⭐⭐)

- Ajoutez un job de scan de sécurité (Trivy ou Snyk) sur l'image Docker
- Implémentez un mécanisme de rollback automatique en cas d'échec
- Ajoutez une notification Slack ou email en cas d'échec du pipeline
- Configurez un cache Maven persistant pour accélérer les builds
- Mettez en place un environnement staging avant la production

10. Récapitulatif & Références

Fichiers créés dans cet atelier

Fichier	Rôle
`Dockerfile`	Build multi-stage de l'application Spring Boot
`.github/workflows/deploy.yml`	Pipeline CI/CD complet (test, build, push, deploy)
`docker-compose.yml`	Orchestration des conteneurs en production

Ressources utiles

- Documentation GitHub Actions : https://docs.github.com/en/actions
- Docker Best Practices : https://docs.docker.com/develop/best-practices/
- Spring Boot Docker Guide : https://spring.io/guides/topicals/spring-boot-docker/
- Action appleboy/ssh-action : https://github.com/appleboy/ssh-action
- Action docker/build-push-action : https://github.com/docker/build-push-action

🏆 **Félicitations !**
Vous disposez maintenant d'un pipeline CI/CD complet et professionnel.
Chaque push vers main déclenchera automatiquement les tests, le build Docker et le déploiement en production — sans intervention manuelle.